

Sieci Neuronowe i algorytmy genetyczne - projekt 2026

Temat: Linear Klasyfikacja danych z biblioteką Keras (TensorFlow)

Wybrane zagadnienie: Linear Neural Networks for Regression

Autorzy: Grzegorz Potocki, Tomasz Majchrzak

Wprowadzenie

Niniejszy projekt został opracowany w celu realizacji klasyfikacji obrazów z datasetu `fashion_minst` zintegrowanego z biblioteką `keras`. Kluczowymi krokami niezbędnymi do poprawnego przeprowadzenia treningu sieci neuronowej (w tym wypadku zgodnie z wybranym zagadnieniem jest to liniowa sieć neuronowa) oraz oceny rezultatów jest odpowiednia analiza eksploracyjna EDA (ang. Exploratory Data Analysis) oraz etap preprocessingu danych.

Wybrany datasetem jest właśnie `fashion_minst`, który zawiera łącznie 70 000 obrazów różnych ubrań i akcesoriów przeznaczonych do przeprowadzenia klasyfikacji.

Do wczytania oraz eksploracyjnej analizy danych zostaną wykorzystane takie moduły jak `numpy`, `pandas`, `seaborn`, `matplotlib` oraz później `pillow` w celu zapisania błędnych klasyfikacji do odpowiednich folderów. Głównymi narzędziami potrzebnymi do zbudowania, treningu oraz testowania sieci neuronowych będą moduły `tensorflow` oraz `keras`.

Projekt skupia się na porównaniu wydajności ze względu na zmianę konkretnych parametrów liniowej sieci neuronowej oraz nieliniowej DNN (ang. Deep Neural Network) z warstwami ukrytymi.

Konfiguracja środowiska

Pierwszym obowiązkowym krokiem w procesie budowania i ewaluacji sieci neuronowych jest poprawna konfiguracja środowiska, w którym będzie się pracować. W tym celu za pomocą modułu `os` zostały wyłączone potencjalne fałszywe komunikaty związane z błędami instalacji modułu `keras` oraz `tensorflow` na systemie linux. Ponadto zaimplementowano małą poprawkę do modułu `numpy` - wyświetlanie liczb zmiennoprzecinkowych z precyzją tylko do trzech liczb po przecinku oraz wyłączenie notacji naukowej dla małych liczb.

```
In [19]: import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"
os.environ["CUDA_VISIBLE_DEVICES"] = "0"
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from PIL import Image

np.set_printoptions(precision=3, suppress=True)
```

```
In [20]: import tensorflow as tf
import keras
from keras import layers
```

```
In [21]: import tensorflow as tf
print("Czy TF widzi GPU?:", tf.config.list_physical_devices('GPU'))
print("Urządzenie logowania operacji:", tf.debugging.get_log_device_place
```

Czy TF widzi GPU?: []
Urządzenie logowania operacji: False

W tym miejscu w ramach utrzymania przejrzystości kodu zostają wyświetlone użyte wersje modułów `tensorflow` oraz `keras`.

```
In [22]: print(tf.__version__, keras.__version__)
```

2.21.0 3.14.1

Zgodnie z przyjętą konwencją podczas pracy z sieciami neuronowymi zostały również zdefiniowane wartości `set_seed` dla `tensorflow` oraz `seed` dla `numpy` w ramach powtarzalności wyników.

```
In [23]: tf.random.set_seed(42)
np.random.seed(42)
```

Eksploracyjna analiza danych (EDA)

Ze względu na to, że dataset jest już zintegrowany z modulem `keras` wystarczy tylko go załadować do odpowiednich zmiennych. Dane zawarte w tym module zostały w łatwy sposób można posortować na treningowe i testowe oraz nie wymagają żadnego potencjalnego pozbywania się wartości `NaN` jak to mogłoby się przydarzyć w zewnętrznych datasetach.

```
In [24]: # Ładowanie zbioru danych
(train_images, train_labels), (test_images, test_labels) = tf.keras.datas

# Definicje nazw klas
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sa
```

Po załadowaniu danych warto sprawdzić jaki kształt mają teraz poszczególne zmienne.

```
In [25]: # Kształty zbiorów treningowych
print("Zbiór treningowy - obrazy:", train_images.shape)
print("Zbiór treningowy - etykiety:", train_labels.shape)
```

```
print("Zbiór testowy - obrazy:", test_images.shape)
print("Zbiór testowy - etykiety:", test_labels.shape)
```

```
Zbiór treningowy - obrazy: (60000, 28, 28)
Zbiór treningowy - etykiety: (60000,)
Zbiór testowy - obrazy: (10000, 28, 28)
Zbiór testowy - etykiety: (10000,)
```

Pomimo faktu, że dane są prawie od razu gotowe do pracy z sieciami neuronowymi to i tak trzeba je poddać normalizacji. Zanim jednak się do tego przejdzie, należy sprawdzić jaki typ oraz jakie są oryginalne wartości, aby późniejsza normalizacja była przeprowadzona w dobry sposób.

```
In [26]: # Typ i zakres wartości przed normalizacją
print("Typ danych:", train_images.dtype)
print("Min wartość piksela:", train_images.min())
print("Max wartość piksela:", train_images.max())
```

```
Typ danych: uint8
Min wartość piksela: 0
Max wartość piksela: 255
```

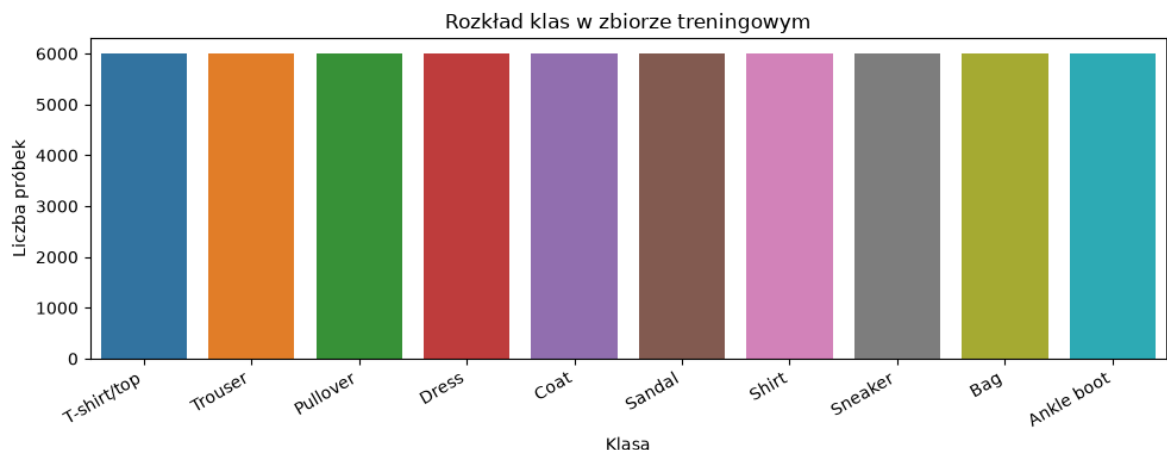
Równie istotną sprawą oprócz samego zakresu wartości danych jest poznanie ich rozkładu.

```
In [27]: unique, counts = np.unique(train_labels, return_counts=True)

class_labels = [class_names[i] for i in unique]

plt.figure(figsize=(10, 4))
sns.barplot(x=class_labels, y=counts, hue=class_labels, legend=False)
plt.title("Rozkład klas w zbiorze treningowym")
plt.xlabel("Klasa")
plt.ylabel("Liczba próbek")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

for name, count in zip(class_labels, counts):
    print(f"{name}: {count} próbek")
```



T-shirt/top: 6000 próbek
 Trouser: 6000 próbek
 Pullover: 6000 próbek
 Dress: 6000 próbek
 Coat: 6000 próbek
 Sandal: 6000 próbek
 Shirt: 6000 próbek
 Sneaker: 6000 próbek
 Bag: 6000 próbek
 Ankle boot: 6000 próbek

W tym miejscu już wszystkie najważniejsze informacje z perspektywy budowania sieci neuronowych są już wyświetlone, ale w ramach dodatkowej wizualizacji można wyświetlić po jednej przykładowej próbce z każdej klasy.

```
In [28]: # Wizualna analiza zbioru danych
# Po jednej próbce z każdej z 10 klas
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
fig.suptitle("Przykładowe próbki ze zbioru Fashion MNIST")

for i, ax in enumerate(axes.flat):
    idx = np.random.choice(np.where(train_labels == i)[0])
    ax.imshow(train_images[idx], cmap="binary")
    ax.set_title(class_names[i], fontsize=9)
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Preprocessing i podział danych

Ze względu na to że, załadowany dataset w zasadzie jest gotowy do pracy jedyną rzeczą, która jest wymagana to znormalizowanie wartości pikseli.

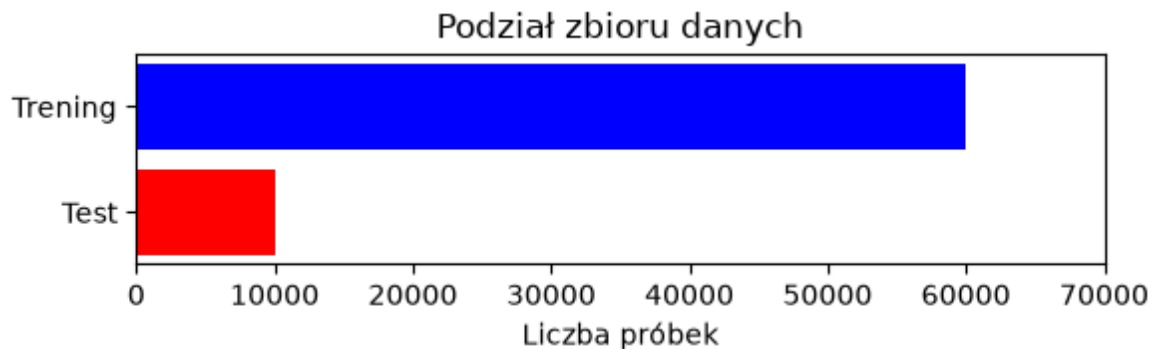
```
In [29]: # Normalizacja wartości pikseli
train_images_normalized = train_images / 255.0
test_images_normalized = test_images / 255.0
```

```
In [30]: # Zakres wartości po normalizacji
print("Min wartość piksela:", train_images_normalized.min())
print("Max wartość piksela:", train_images_normalized.max())
```

Min wartość piksela: 0.0

Max wartość piksela: 1.0

```
In [31]: # Podział zbioru danych
plt.figure(figsize=(6, 2))
bars = plt.barh(["Test", "Trening"], [10000, 60000], color=["red", "blue"])
plt.xlabel("Liczba próbek")
plt.xlim(0, 70000)
plt.title("Podział zbioru danych")
plt.tight_layout()
plt.show()
```



Budowa i trening modeli oraz ich ewaluacja

W tym miejscu zostaną przeprowadzone testy na liniowej sieci. W tym celu zostały zdefiniowane następujące funkcje:

- `build_linear_model()` - funkcja budująca prosty model liniowy za pomocą `keras.Sequential()`. Obraz jest spłaszczony do pojedynczego wektora, a warstwa wyjściowa przypisuje prawdopodobieństwo każdej z klas.
- `plot_history()` - funkcja odpowiedzialna za wyświetlanie dwóch wykresów - pierwszy przedstawia jak zmienia się wartość wskaźnika accuracy względem postępujących epok, a drugi to charakterystyka wartości funkcji błędu względem epok
- `build_dnn_model()` - budowanie głębokiej sieci neuronowej z dwoma warstwami głębokimi, aktywowanymi za pomocą funkcji `relu`
- `run_experiment()` - funkcja pomocnicza, zbierająca w całość całą procedurę testową

```
In [32]: all_results = []
```

```
In [33]: # Model liniowy: Flatten spłaszcza obraz 28x28 do wektora 784 pikseli
# Warstwa Dense(10, softmax) przypisuje prawdopodobieństwo każdej z 10 kl
def build_linear_model(lr):
    model = keras.Sequential([
        keras.Input(shape=(28,28)),
        layers.Flatten(),
        layers.Dense(10, activation="softmax")
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
```

```
)
return model
```

```
In [34]: # Krzywe uczenia - dokładność na zbiorze treningowym
# i walidacyjnym w funkcji epoki
def plot_history(history, title):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle(title)

    # Krzywa accuracy
    axes[0].plot(history.history["accuracy"], label="trening")
    axes[0].plot(history.history["val_accuracy"], label="walidacja")
    axes[0].set_xlabel("Epoka")
    axes[0].set_ylabel("Accuracy")
    axes[0].set_ylim([0, 1])
    axes[0].legend()
    axes[0].grid(True)

    # Krzywa loss
    axes[1].plot(history.history["loss"], label="trening")
    axes[1].plot(history.history["val_loss"], label="walidacja")
    axes[1].set_xlabel("Epoka")
    axes[1].set_ylabel("Loss")
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
```

```
In [35]: def run_experiment(build_fn, model_name, lr, epochs, results, save_errors):
    model = build_fn(lr)

    if show_summary:
        model.summary()

    history = model.fit(
        train_images_normalized, train_labels,
        epochs=epochs,
        validation_split=0.1,
        verbose=1
    )

    _, acc = model.evaluate(test_images_normalized, test_labels, verbose=
    plot_history(history, title=f"{model_name} | epochs={epochs} | lr={lr}

    y_pred_proba = model.predict(test_images_normalized, verbose=1)
    y_pred = np.argmax(y_pred_proba, axis=1)

    if save_errors:
        folder = f"wrong_predictions/{model_name}_epochs={epochs}_lr={lr}"
        os.makedirs(folder, exist_ok=True)
        wrong_idx = np.where(y_pred != test_labels)[0]
        for idx in wrong_idx:
            img = Image.fromarray((test_images_normalized[idx] * 255).ast
            true = class_names[test_labels[idx]].replace("/", "-")
            pred = class_names[y_pred[idx]].replace("/", "-")
            conf = y_pred_proba[idx, y_pred[idx]]
            img.save(f"{folder}/{idx}_true={true}_pred={pred}_conf={conf:
            print(f"Zapisano {len(wrong_idx)} błędów -> {folder}/")
```

```
results.append({"Model": model_name, "Epochs": epochs, "Learning rate
```

```
In [36]: # Model liniowy - wpływ liczby epok
for i, epochs in enumerate([10, 20, 50]):
    run_experiment(build_linear_model,
                  "Liniowy",
                  lr=0.001,
                  epochs=epochs,
                  results=all_results,
                  show_summary=(i == 0))
```

Model: "sequential_1"

Layer (type)	Output Shape	
flatten_1 (Flatten)	(None, 784)	
dense_1 (Dense)	(None, 10)	



Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

1688/1688 ————— 2s 955us/step - accuracy: 0.7934 - loss: 0.6122 - val_accuracy: 0.8283 - val_loss: 0.4922

Epoch 2/10

1688/1688 ————— 2s 903us/step - accuracy: 0.8406 - loss: 0.4679 - val_accuracy: 0.8387 - val_loss: 0.4586

Epoch 3/10

1688/1688 ————— 2s 1ms/step - accuracy: 0.8487 - loss: 0.406 - val_accuracy: 0.8422 - val_loss: 0.4449

Epoch 4/10

1688/1688 ————— 1s 754us/step - accuracy: 0.8524 - loss: 0.4260 - val_accuracy: 0.8452 - val_loss: 0.4370

Epoch 5/10

1688/1688 ————— 1s 745us/step - accuracy: 0.8559 - loss: 0.4166 - val_accuracy: 0.8485 - val_loss: 0.4318

Epoch 6/10

1688/1688 ————— 1s 750us/step - accuracy: 0.8582 - loss: 0.4097 - val_accuracy: 0.8493 - val_loss: 0.4281

Epoch 7/10

1688/1688 ————— 1s 780us/step - accuracy: 0.8604 - loss: 0.4044 - val_accuracy: 0.8503 - val_loss: 0.4254

Epoch 8/10

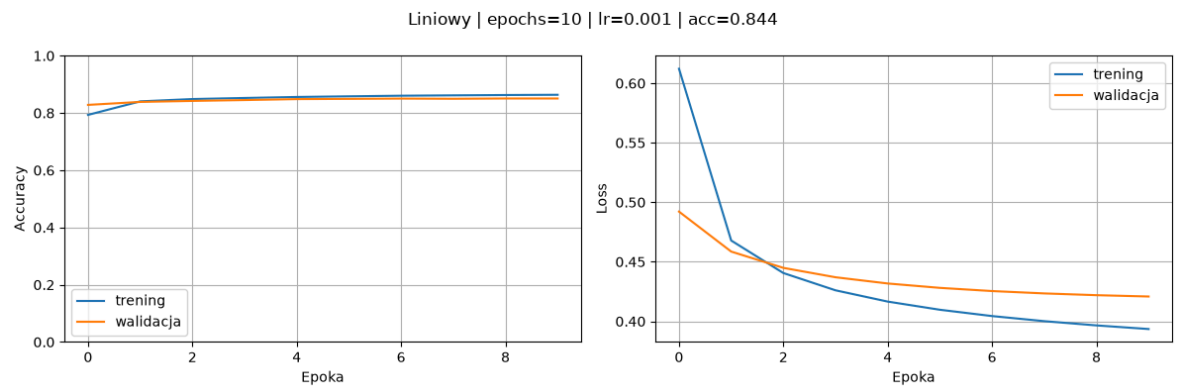
1688/1688 ————— 1s 688us/step - accuracy: 0.8616 - loss: 0.4001 - val_accuracy: 0.8497 - val_loss: 0.4234

Epoch 9/10

1688/1688 ————— 1s 761us/step - accuracy: 0.8627 - loss: 0.3966 - val_accuracy: 0.8507 - val_loss: 0.4220

Epoch 10/10

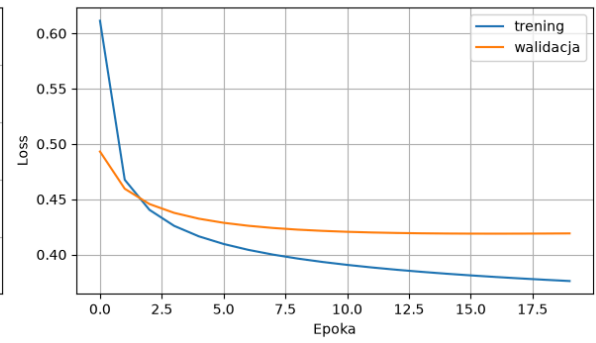
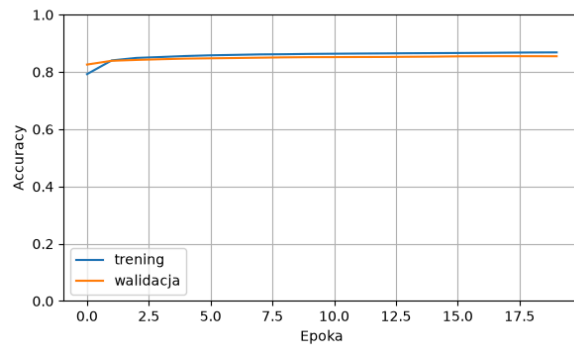
1688/1688 ————— 1s 808us/step - accuracy: 0.8636 - loss: 0.3935 - val_accuracy: 0.8507 - val_loss: 0.4208



313/313 ————— **0s** 392us/step
Zapisano 1565 błędów -> wrong_predictions/Liniowy_epochs=10_lr=0.001/
Epoch 1/20
1688/1688 ————— **2s** 806us/step - accuracy: 0.7926 - loss: 0.
6114 - val_accuracy: 0.8260 - val_loss: 0.4932
Epoch 2/20
1688/1688 ————— **1s** 701us/step - accuracy: 0.8405 - loss: 0.
4676 - val_accuracy: 0.8392 - val_loss: 0.4595
Epoch 3/20
1688/1688 ————— **1s** 718us/step - accuracy: 0.8492 - loss: 0.
4404 - val_accuracy: 0.8422 - val_loss: 0.4457
Epoch 4/20
1688/1688 ————— **1s** 673us/step - accuracy: 0.8524 - loss: 0.
4259 - val_accuracy: 0.8447 - val_loss: 0.4377
Epoch 5/20
1688/1688 ————— **1s** 687us/step - accuracy: 0.8559 - loss: 0.
4165 - val_accuracy: 0.8468 - val_loss: 0.4324
Epoch 6/20
1688/1688 ————— **1s** 666us/step - accuracy: 0.8583 - loss: 0.
4096 - val_accuracy: 0.8478 - val_loss: 0.4287
Epoch 7/20
1688/1688 ————— **1s** 754us/step - accuracy: 0.8599 - loss: 0.
4043 - val_accuracy: 0.8488 - val_loss: 0.4260
Epoch 8/20
1688/1688 ————— **1s** 673us/step - accuracy: 0.8615 - loss: 0.
4000 - val_accuracy: 0.8500 - val_loss: 0.4240
Epoch 9/20
1688/1688 ————— **1s** 678us/step - accuracy: 0.8621 - loss: 0.
3964 - val_accuracy: 0.8512 - val_loss: 0.4225
Epoch 10/20
1688/1688 ————— **1s** 716us/step - accuracy: 0.8632 - loss: 0.
3933 - val_accuracy: 0.8518 - val_loss: 0.4214
Epoch 11/20
1688/1688 ————— **1s** 673us/step - accuracy: 0.8639 - loss: 0.
3907 - val_accuracy: 0.8520 - val_loss: 0.4206
Epoch 12/20
1688/1688 ————— **1s** 676us/step - accuracy: 0.8645 - loss: 0.
3883 - val_accuracy: 0.8523 - val_loss: 0.4200
Epoch 13/20
1688/1688 ————— **1s** 668us/step - accuracy: 0.8650 - loss: 0.
3863 - val_accuracy: 0.8525 - val_loss: 0.4195
Epoch 14/20
1688/1688 ————— **1s** 682us/step - accuracy: 0.8656 - loss: 0.
3844 - val_accuracy: 0.8532 - val_loss: 0.4192
Epoch 15/20
1688/1688 ————— **1s** 668us/step - accuracy: 0.8660 - loss: 0.
3827 - val_accuracy: 0.8537 - val_loss: 0.4190
Epoch 16/20
1688/1688 ————— **1s** 702us/step - accuracy: 0.8664 - loss: 0.
3812 - val_accuracy: 0.8547 - val_loss: 0.4189
Epoch 17/20
1688/1688 ————— **1s** 678us/step - accuracy: 0.8669 - loss: 0.
3797 - val_accuracy: 0.8550 - val_loss: 0.4189
Epoch 18/20
1688/1688 ————— **1s** 663us/step - accuracy: 0.8676 - loss: 0.
3784 - val_accuracy: 0.8552 - val_loss: 0.4189
Epoch 19/20
1688/1688 ————— **1s** 678us/step - accuracy: 0.8682 - loss: 0.
3772 - val_accuracy: 0.8552 - val_loss: 0.4190
Epoch 20/20

1688/1688 ————— **1s** 665us/step - accuracy: 0.8685 - loss: 0.3761 - val_accuracy: 0.8550 - val_loss: 0.4191

Liniowy | epochs=20 | lr=0.001 | acc=0.844



313/313 ————— **0s** 392us/step
Zapisano 1558 błędów -> wrong_predictions/Liniowy_epochs=20_lr=0.001/
Epoch 1/50
1688/1688 ————— **2s** 895us/step - accuracy: 0.7963 - loss: 0.
6076 - val_accuracy: 0.8260 - val_loss: 0.4922
Epoch 2/50
1688/1688 ————— **1s** 664us/step - accuracy: 0.8408 - loss: 0.
4674 - val_accuracy: 0.8397 - val_loss: 0.4590
Epoch 3/50
1688/1688 ————— **1s** 670us/step - accuracy: 0.8495 - loss: 0.
4405 - val_accuracy: 0.8442 - val_loss: 0.4452
Epoch 4/50
1688/1688 ————— **1s** 678us/step - accuracy: 0.8538 - loss: 0.
4260 - val_accuracy: 0.8470 - val_loss: 0.4372
Epoch 5/50
1688/1688 ————— **1s** 701us/step - accuracy: 0.8566 - loss: 0.
4166 - val_accuracy: 0.8482 - val_loss: 0.4319
Epoch 6/50
1688/1688 ————— **1s** 688us/step - accuracy: 0.8588 - loss: 0.
4097 - val_accuracy: 0.8487 - val_loss: 0.4282
Epoch 7/50
1688/1688 ————— **1s** 696us/step - accuracy: 0.8604 - loss: 0.
4044 - val_accuracy: 0.8492 - val_loss: 0.4255
Epoch 8/50
1688/1688 ————— **1s** 676us/step - accuracy: 0.8617 - loss: 0.
4001 - val_accuracy: 0.8497 - val_loss: 0.4235
Epoch 9/50
1688/1688 ————— **1s** 674us/step - accuracy: 0.8626 - loss: 0.
3965 - val_accuracy: 0.8505 - val_loss: 0.4221
Epoch 10/50
1688/1688 ————— **1s** 673us/step - accuracy: 0.8634 - loss: 0.
3935 - val_accuracy: 0.8508 - val_loss: 0.4210
Epoch 11/50
1688/1688 ————— **1s** 689us/step - accuracy: 0.8641 - loss: 0.
3909 - val_accuracy: 0.8520 - val_loss: 0.4201
Epoch 12/50
1688/1688 ————— **1s** 675us/step - accuracy: 0.8645 - loss: 0.
3885 - val_accuracy: 0.8522 - val_loss: 0.4195
Epoch 13/50
1688/1688 ————— **1s** 683us/step - accuracy: 0.8650 - loss: 0.
3864 - val_accuracy: 0.8527 - val_loss: 0.4191
Epoch 14/50
1688/1688 ————— **1s** 691us/step - accuracy: 0.8656 - loss: 0.
3846 - val_accuracy: 0.8527 - val_loss: 0.4188
Epoch 15/50
1688/1688 ————— **1s** 685us/step - accuracy: 0.8661 - loss: 0.
3829 - val_accuracy: 0.8532 - val_loss: 0.4186
Epoch 16/50
1688/1688 ————— **1s** 676us/step - accuracy: 0.8668 - loss: 0.
3813 - val_accuracy: 0.8533 - val_loss: 0.4185
Epoch 17/50
1688/1688 ————— **1s** 662us/step - accuracy: 0.8673 - loss: 0.
3799 - val_accuracy: 0.8533 - val_loss: 0.4184
Epoch 18/50
1688/1688 ————— **1s** 665us/step - accuracy: 0.8676 - loss: 0.
3786 - val_accuracy: 0.8535 - val_loss: 0.4185
Epoch 19/50
1688/1688 ————— **1s** 669us/step - accuracy: 0.8678 - loss: 0.
3774 - val_accuracy: 0.8537 - val_loss: 0.4185
Epoch 20/50

1688/1688 ————— **1s** 700us/step - accuracy: 0.8683 - loss: 0.3762 - val_accuracy: 0.8535 - val_loss: 0.4187
Epoch 21/50

1688/1688 ————— **1s** 725us/step - accuracy: 0.8685 - loss: 0.3752 - val_accuracy: 0.8537 - val_loss: 0.4188
Epoch 22/50

1688/1688 ————— **1s** 666us/step - accuracy: 0.8688 - loss: 0.3742 - val_accuracy: 0.8537 - val_loss: 0.4190
Epoch 23/50

1688/1688 ————— **1s** 657us/step - accuracy: 0.8691 - loss: 0.3732 - val_accuracy: 0.8538 - val_loss: 0.4192
Epoch 24/50

1688/1688 ————— **1s** 670us/step - accuracy: 0.8693 - loss: 0.3723 - val_accuracy: 0.8535 - val_loss: 0.4195
Epoch 25/50

1688/1688 ————— **1s** 667us/step - accuracy: 0.8694 - loss: 0.3715 - val_accuracy: 0.8540 - val_loss: 0.4197
Epoch 26/50

1688/1688 ————— **1s** 668us/step - accuracy: 0.8697 - loss: 0.3707 - val_accuracy: 0.8538 - val_loss: 0.4200
Epoch 27/50

1688/1688 ————— **1s** 664us/step - accuracy: 0.8700 - loss: 0.3699 - val_accuracy: 0.8543 - val_loss: 0.4203
Epoch 28/50

1688/1688 ————— **1s** 699us/step - accuracy: 0.8702 - loss: 0.3692 - val_accuracy: 0.8542 - val_loss: 0.4206
Epoch 29/50

1688/1688 ————— **1s** 764us/step - accuracy: 0.8704 - loss: 0.3685 - val_accuracy: 0.8538 - val_loss: 0.4209
Epoch 30/50

1688/1688 ————— **1s** 711us/step - accuracy: 0.8708 - loss: 0.3678 - val_accuracy: 0.8535 - val_loss: 0.4212
Epoch 31/50

1688/1688 ————— **1s** 702us/step - accuracy: 0.8712 - loss: 0.3672 - val_accuracy: 0.8537 - val_loss: 0.4216
Epoch 32/50

1688/1688 ————— **1s** 708us/step - accuracy: 0.8715 - loss: 0.3666 - val_accuracy: 0.8537 - val_loss: 0.4219
Epoch 33/50

1688/1688 ————— **1s** 733us/step - accuracy: 0.8717 - loss: 0.3660 - val_accuracy: 0.8532 - val_loss: 0.4222
Epoch 34/50

1688/1688 ————— **1s** 753us/step - accuracy: 0.8720 - loss: 0.3654 - val_accuracy: 0.8530 - val_loss: 0.4226
Epoch 35/50

1688/1688 ————— **1s** 823us/step - accuracy: 0.8722 - loss: 0.3649 - val_accuracy: 0.8535 - val_loss: 0.4229
Epoch 36/50

1688/1688 ————— **1s** 823us/step - accuracy: 0.8724 - loss: 0.3643 - val_accuracy: 0.8537 - val_loss: 0.4233
Epoch 37/50

1688/1688 ————— **1s** 810us/step - accuracy: 0.8724 - loss: 0.3638 - val_accuracy: 0.8535 - val_loss: 0.4237
Epoch 38/50

1688/1688 ————— **1s** 686us/step - accuracy: 0.8726 - loss: 0.3633 - val_accuracy: 0.8532 - val_loss: 0.4240
Epoch 39/50

1688/1688 ————— **1s** 700us/step - accuracy: 0.8728 - loss: 0.3629 - val_accuracy: 0.8528 - val_loss: 0.4244
Epoch 40/50

1688/1688 ————— 1s 743us/step - accuracy: 0.8730 - loss: 0.3624 - val_accuracy: 0.8522 - val_loss: 0.4248
Epoch 41/50

1688/1688 ————— 1s 779us/step - accuracy: 0.8731 - loss: 0.3620 - val_accuracy: 0.8523 - val_loss: 0.4251
Epoch 42/50

1688/1688 ————— 1s 741us/step - accuracy: 0.8732 - loss: 0.3615 - val_accuracy: 0.8527 - val_loss: 0.4255
Epoch 43/50

1688/1688 ————— 1s 754us/step - accuracy: 0.8735 - loss: 0.3611 - val_accuracy: 0.8527 - val_loss: 0.4259
Epoch 44/50

1688/1688 ————— 1s 754us/step - accuracy: 0.8734 - loss: 0.3607 - val_accuracy: 0.8525 - val_loss: 0.4263
Epoch 45/50

1688/1688 ————— 1s 740us/step - accuracy: 0.8737 - loss: 0.3603 - val_accuracy: 0.8527 - val_loss: 0.4266
Epoch 46/50

1688/1688 ————— 1s 735us/step - accuracy: 0.8738 - loss: 0.3599 - val_accuracy: 0.8527 - val_loss: 0.4270
Epoch 47/50

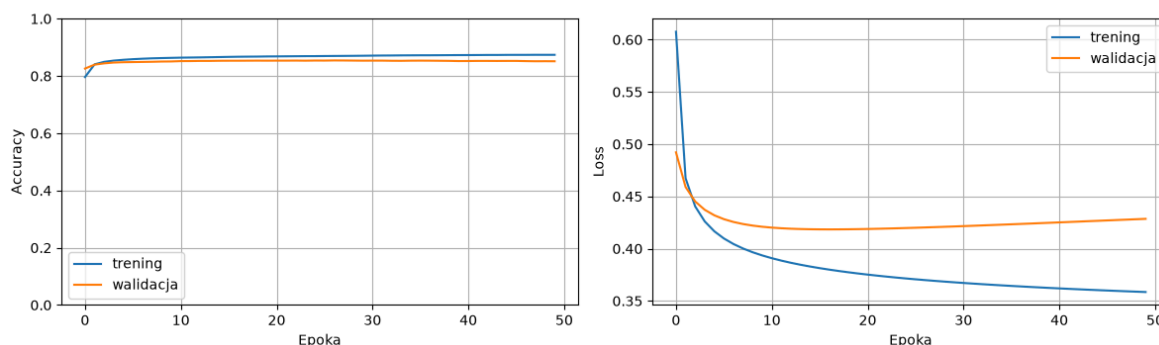
1688/1688 ————— 1s 730us/step - accuracy: 0.8738 - loss: 0.3596 - val_accuracy: 0.8520 - val_loss: 0.4274
Epoch 48/50

1688/1688 ————— 1s 759us/step - accuracy: 0.8740 - loss: 0.3592 - val_accuracy: 0.8515 - val_loss: 0.4278
Epoch 49/50

1688/1688 ————— 1s 773us/step - accuracy: 0.8739 - loss: 0.3588 - val_accuracy: 0.8517 - val_loss: 0.4281
Epoch 50/50

1688/1688 ————— 1s 766us/step - accuracy: 0.8739 - loss: 0.3585 - val_accuracy: 0.8515 - val_loss: 0.4285

Liniowy | epochs=50 | lr=0.001 | acc=0.843



313/313 ————— 0s 395us/step

Zapisano 1571 błędów -> wrong_predictions/Liniowy_epochs=50_lr=0.001/

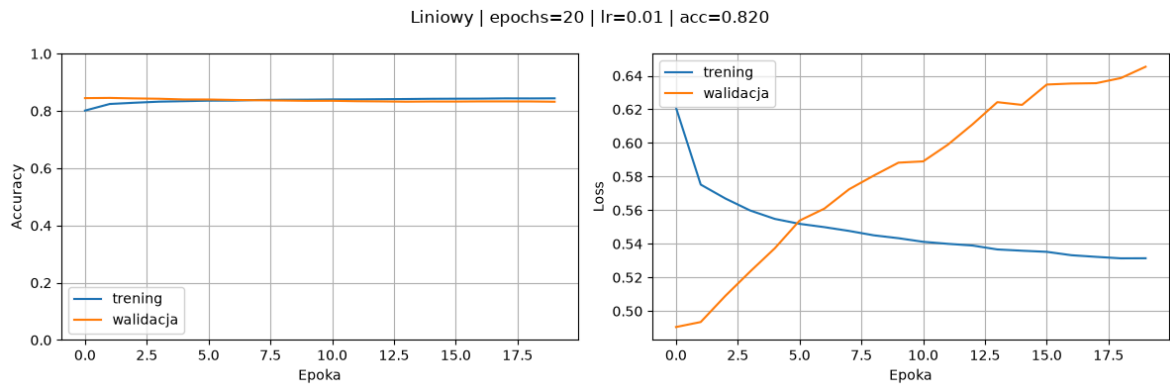
```
In [37]: # Model liniowy - wpływ learning rate
for lr in [0.01, 0.001, 0.0001]:
    run_experiment(build_linear_model, "Liniowy", lr=lr, epochs=20, resul
```

Model: "sequential_4"

Layer (type)	Output Shape	
flatten_4 (Flatten)	(None, 784)	
dense_4 (Dense)	(None, 10)	

Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688 ————— **2s** 785us/step - accuracy: 0.8014 - loss: 0.6210 - val_accuracy: 0.8450 - val_loss: 0.4905
Epoch 2/20
1688/1688 ————— **1s** 723us/step - accuracy: 0.8245 - loss: 0.5753 - val_accuracy: 0.8457 - val_loss: 0.4935
Epoch 3/20
1688/1688 ————— **1s** 667us/step - accuracy: 0.8288 - loss: 0.5670 - val_accuracy: 0.8440 - val_loss: 0.5091
Epoch 4/20
1688/1688 ————— **1s** 673us/step - accuracy: 0.8326 - loss: 0.5598 - val_accuracy: 0.8428 - val_loss: 0.5235
Epoch 5/20
1688/1688 ————— **1s** 683us/step - accuracy: 0.8343 - loss: 0.5548 - val_accuracy: 0.8403 - val_loss: 0.5374
Epoch 6/20
1688/1688 ————— **1s** 685us/step - accuracy: 0.8360 - loss: 0.5519 - val_accuracy: 0.8402 - val_loss: 0.5538
Epoch 7/20
1688/1688 ————— **1s** 667us/step - accuracy: 0.8363 - loss: 0.5499 - val_accuracy: 0.8387 - val_loss: 0.5609
Epoch 8/20
1688/1688 ————— **1s** 682us/step - accuracy: 0.8385 - loss: 0.5477 - val_accuracy: 0.8375 - val_loss: 0.5724
Epoch 9/20
1688/1688 ————— **1s** 676us/step - accuracy: 0.8394 - loss: 0.5451 - val_accuracy: 0.8365 - val_loss: 0.5806
Epoch 10/20
1688/1688 ————— **1s** 666us/step - accuracy: 0.8396 - loss: 0.5434 - val_accuracy: 0.8355 - val_loss: 0.5883
Epoch 11/20
1688/1688 ————— **1s** 676us/step - accuracy: 0.8404 - loss: 0.5412 - val_accuracy: 0.8357 - val_loss: 0.5891
Epoch 12/20
1688/1688 ————— **1s** 687us/step - accuracy: 0.8406 - loss: 0.5400 - val_accuracy: 0.8342 - val_loss: 0.5990
Epoch 13/20
1688/1688 ————— **1s** 702us/step - accuracy: 0.8412 - loss: 0.5390 - val_accuracy: 0.8335 - val_loss: 0.6111
Epoch 14/20
1688/1688 ————— **1s** 671us/step - accuracy: 0.8419 - loss: 0.5367 - val_accuracy: 0.8325 - val_loss: 0.6243
Epoch 15/20
1688/1688 ————— **1s** 681us/step - accuracy: 0.8427 - loss: 0.5359 - val_accuracy: 0.8332 - val_loss: 0.6227
Epoch 16/20
1688/1688 ————— **1s** 676us/step - accuracy: 0.8430 - loss: 0.5353 - val_accuracy: 0.8330 - val_loss: 0.6348
Epoch 17/20
1688/1688 ————— **1s** 673us/step - accuracy: 0.8432 - loss: 0.5332 - val_accuracy: 0.8335 - val_loss: 0.6354
Epoch 18/20
1688/1688 ————— **1s** 671us/step - accuracy: 0.8441 - loss: 0.5323 - val_accuracy: 0.8335 - val_loss: 0.6356
Epoch 19/20
1688/1688 ————— **1s** 722us/step - accuracy: 0.8439 - loss: 0.5314 - val_accuracy: 0.8333 - val_loss: 0.6386
Epoch 20/20
1688/1688 ————— **1s** 669us/step - accuracy: 0.8443 - loss: 0.5314 - val_accuracy: 0.8323 - val_loss: 0.6454



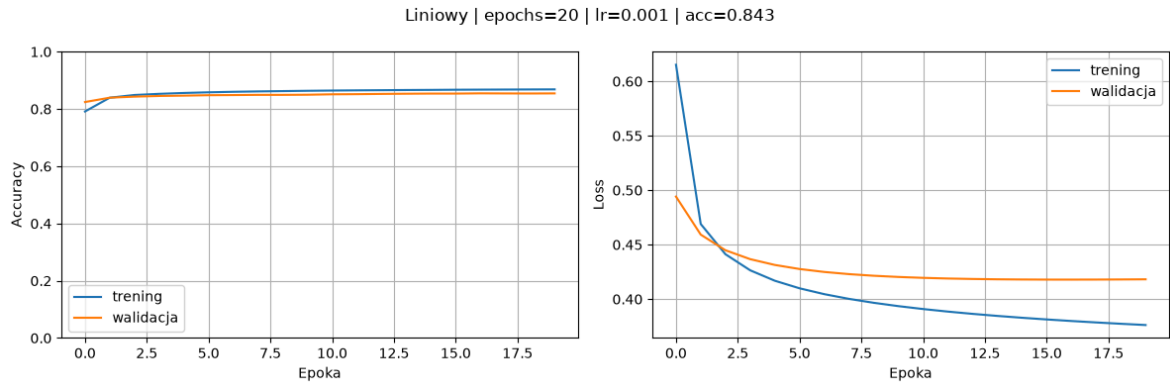
313/313 ————— 0s 408us/step
Zapisano 1799 błędów -> wrong_predictions/Liniowy_epochs=20_lr=0.01/
Model: "sequential_5"

Layer (type)	Output Shape	
flatten_5 (Flatten)	(None, 784)	
dense_5 (Dense)	(None, 10)	



Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688  **2s** 827us/step - accuracy: 0.7914 - loss: 0.6152 - val_accuracy: 0.8243 - val_loss: 0.4941
Epoch 2/20
1688/1688  **1s** 750us/step - accuracy: 0.8394 - loss: 0.4690 - val_accuracy: 0.8395 - val_loss: 0.4593
Epoch 3/20
1688/1688  **1s** 681us/step - accuracy: 0.8489 - loss: 0.4412 - val_accuracy: 0.8432 - val_loss: 0.4449
Epoch 4/20
1688/1688  **1s** 698us/step - accuracy: 0.8529 - loss: 0.4265 - val_accuracy: 0.8455 - val_loss: 0.4367
Epoch 5/20
1688/1688  **1s** 671us/step - accuracy: 0.8560 - loss: 0.4169 - val_accuracy: 0.8467 - val_loss: 0.4314
Epoch 6/20
1688/1688  **1s** 679us/step - accuracy: 0.8583 - loss: 0.4099 - val_accuracy: 0.8482 - val_loss: 0.4276
Epoch 7/20
1688/1688  **1s** 698us/step - accuracy: 0.8600 - loss: 0.4045 - val_accuracy: 0.8487 - val_loss: 0.4249
Epoch 8/20
1688/1688  **1s** 688us/step - accuracy: 0.8614 - loss: 0.4002 - val_accuracy: 0.8493 - val_loss: 0.4229
Epoch 9/20
1688/1688  **1s** 709us/step - accuracy: 0.8625 - loss: 0.3966 - val_accuracy: 0.8493 - val_loss: 0.4215
Epoch 10/20
1688/1688  **1s** 695us/step - accuracy: 0.8635 - loss: 0.3935 - val_accuracy: 0.8498 - val_loss: 0.4203
Epoch 11/20
1688/1688  **1s** 722us/step - accuracy: 0.8644 - loss: 0.3909 - val_accuracy: 0.8513 - val_loss: 0.4195
Epoch 12/20
1688/1688  **1s** 687us/step - accuracy: 0.8651 - loss: 0.3885 - val_accuracy: 0.8518 - val_loss: 0.4189
Epoch 13/20
1688/1688  **1s** 690us/step - accuracy: 0.8657 - loss: 0.3865 - val_accuracy: 0.8527 - val_loss: 0.4185
Epoch 14/20
1688/1688  **1s** 714us/step - accuracy: 0.8663 - loss: 0.3846 - val_accuracy: 0.8533 - val_loss: 0.4182
Epoch 15/20
1688/1688  **1s** 679us/step - accuracy: 0.8667 - loss: 0.3829 - val_accuracy: 0.8538 - val_loss: 0.4180
Epoch 16/20
1688/1688  **1s** 680us/step - accuracy: 0.8673 - loss: 0.3813 - val_accuracy: 0.8538 - val_loss: 0.4179
Epoch 17/20
1688/1688  **1s** 704us/step - accuracy: 0.8676 - loss: 0.3799 - val_accuracy: 0.8547 - val_loss: 0.4179
Epoch 18/20
1688/1688  **1s** 687us/step - accuracy: 0.8679 - loss: 0.3786 - val_accuracy: 0.8543 - val_loss: 0.4180
Epoch 19/20
1688/1688  **1s** 664us/step - accuracy: 0.8682 - loss: 0.3774 - val_accuracy: 0.8542 - val_loss: 0.4181
Epoch 20/20
1688/1688  **1s** 691us/step - accuracy: 0.8685 - loss: 0.3762 - val_accuracy: 0.8545 - val_loss: 0.4182



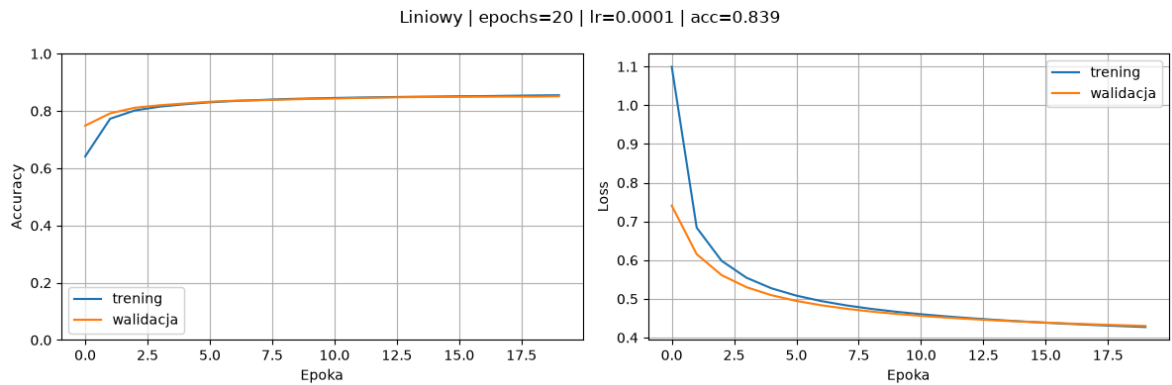
313/313 ————— 0s 401us/step
Zapisano 1568 błędów -> wrong_predictions/Liniowy_epochs=20_lr=0.001/
Model: "sequential_6"

Layer (type)	Output Shape	
flatten_6 (Flatten)	(None, 784)	
dense_6 (Dense)	(None, 10)	



Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688 ————— **2s** 824us/step - accuracy: 0.6415 - loss: 1.0996 - val_accuracy: 0.7490 - val_loss: 0.7408
Epoch 2/20
1688/1688 ————— **1s** 737us/step - accuracy: 0.7730 - loss: 0.6840 - val_accuracy: 0.7918 - val_loss: 0.6155
Epoch 3/20
1688/1688 ————— **1s** 717us/step - accuracy: 0.8022 - loss: 0.5983 - val_accuracy: 0.8115 - val_loss: 0.5613
Epoch 4/20
1688/1688 ————— **1s** 710us/step - accuracy: 0.8154 - loss: 0.5545 - val_accuracy: 0.8202 - val_loss: 0.5301
Epoch 5/20
1688/1688 ————— **1s** 681us/step - accuracy: 0.8237 - loss: 0.5273 - val_accuracy: 0.8263 - val_loss: 0.5095
Epoch 6/20
1688/1688 ————— **1s** 692us/step - accuracy: 0.8305 - loss: 0.5083 - val_accuracy: 0.8323 - val_loss: 0.4947
Epoch 7/20
1688/1688 ————— **1s** 695us/step - accuracy: 0.8351 - loss: 0.4942 - val_accuracy: 0.8360 - val_loss: 0.4834
Epoch 8/20
1688/1688 ————— **1s** 683us/step - accuracy: 0.8385 - loss: 0.4832 - val_accuracy: 0.8378 - val_loss: 0.4745
Epoch 9/20
1688/1688 ————— **1s** 689us/step - accuracy: 0.8416 - loss: 0.4742 - val_accuracy: 0.8400 - val_loss: 0.4673
Epoch 10/20
1688/1688 ————— **1s** 706us/step - accuracy: 0.8438 - loss: 0.4667 - val_accuracy: 0.8423 - val_loss: 0.4612
Epoch 11/20
1688/1688 ————— **1s** 670us/step - accuracy: 0.8456 - loss: 0.4603 - val_accuracy: 0.8438 - val_loss: 0.4561
Epoch 12/20
1688/1688 ————— **1s** 710us/step - accuracy: 0.8471 - loss: 0.4548 - val_accuracy: 0.8457 - val_loss: 0.4517
Epoch 13/20
1688/1688 ————— **1s** 679us/step - accuracy: 0.8483 - loss: 0.4500 - val_accuracy: 0.8472 - val_loss: 0.4478
Epoch 14/20
1688/1688 ————— **1s** 676us/step - accuracy: 0.8496 - loss: 0.4457 - val_accuracy: 0.8485 - val_loss: 0.4444
Epoch 15/20
1688/1688 ————— **1s** 689us/step - accuracy: 0.8504 - loss: 0.4418 - val_accuracy: 0.8492 - val_loss: 0.4414
Epoch 16/20
1688/1688 ————— **1s** 679us/step - accuracy: 0.8515 - loss: 0.4384 - val_accuracy: 0.8495 - val_loss: 0.4386
Epoch 17/20
1688/1688 ————— **1s** 683us/step - accuracy: 0.8521 - loss: 0.4352 - val_accuracy: 0.8500 - val_loss: 0.4362
Epoch 18/20
1688/1688 ————— **1s** 678us/step - accuracy: 0.8532 - loss: 0.4323 - val_accuracy: 0.8503 - val_loss: 0.4340
Epoch 19/20
1688/1688 ————— **1s** 706us/step - accuracy: 0.8543 - loss: 0.4297 - val_accuracy: 0.8502 - val_loss: 0.4320
Epoch 20/20
1688/1688 ————— **1s** 708us/step - accuracy: 0.8551 - loss: 0.4273 - val_accuracy: 0.8515 - val_loss: 0.4301



313/313 ————— 0s 381us/step

Zapisano 1606 błędów -> wrong_predictions/Liniowy_epochs=20_lr=0.0001/

Eksperyment z siecią nieliniową (porównanie)

Następnym etapem ewaluacji zbudowanych liniowych sieci neuronowych jest porównanie ich do przykładowej sieci nieliniowej z dwiema warstwami ukrytymi, aktywowanymi za pomocą funkcji `relu` w ramach wprowadzenia nieliniowości.

```
In [38]: # Model nieliniowy z dwiema ukrytymi warstwami ReLU - punkt odniesienia
# Dwie warstwy Dense(128) i Dense(64) z aktywacją ReLU
# wprowadzają nieliniowość, której model liniowy nie posiada
def build_dnn_model(lr):
    model = keras.Sequential([
        keras.Input(shape=(28,28)),
        layers.Flatten(),
        layers.Dense(128, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

```
In [39]: # DNN – wpływ learning rate
for lr in [0.01, 0.001, 0.0001]:
    run_experiment(build_dnn_model, "DNN", lr=lr, epochs=20, results=all_
```

Model: "sequential_7"

Layer (type)	Output Shape	
flatten_7 (Flatten)	(None, 784)	
dense_7 (Dense)	(None, 128)	
dense_8 (Dense)	(None, 64)	
dense_9 (Dense)	(None, 10)	

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688  3s 2ms/step - accuracy: 0.8048 - loss: 0.54
27 - val_accuracy: 0.8363 - val_loss: 0.4865

Epoch 2/20
1688/1688  2s 1ms/step - accuracy: 0.8420 - loss: 0.44
30 - val_accuracy: 0.8337 - val_loss: 0.4697

Epoch 3/20
1688/1688  2s 1ms/step - accuracy: 0.8504 - loss: 0.42
32 - val_accuracy: 0.8458 - val_loss: 0.4308

Epoch 4/20
1688/1688  2s 1ms/step - accuracy: 0.8553 - loss: 0.40
93 - val_accuracy: 0.8528 - val_loss: 0.4221

Epoch 5/20
1688/1688  2s 1ms/step - accuracy: 0.8593 - loss: 0.39
85 - val_accuracy: 0.8492 - val_loss: 0.4380

Epoch 6/20
1688/1688  2s 1ms/step - accuracy: 0.8627 - loss: 0.38
88 - val_accuracy: 0.8572 - val_loss: 0.4191

Epoch 7/20
1688/1688  2s 1ms/step - accuracy: 0.8629 - loss: 0.38
43 - val_accuracy: 0.8435 - val_loss: 0.4575

Epoch 8/20
1688/1688  2s 1ms/step - accuracy: 0.8618 - loss: 0.38
73 - val_accuracy: 0.8568 - val_loss: 0.4338

Epoch 9/20
1688/1688  2s 1ms/step - accuracy: 0.8672 - loss: 0.37
67 - val_accuracy: 0.8562 - val_loss: 0.4344

Epoch 10/20
1688/1688  2s 1ms/step - accuracy: 0.8685 - loss: 0.37
01 - val_accuracy: 0.8483 - val_loss: 0.4571

Epoch 11/20
1688/1688  2s 1ms/step - accuracy: 0.8697 - loss: 0.36
74 - val_accuracy: 0.8560 - val_loss: 0.4334

Epoch 12/20
1688/1688  2s 1ms/step - accuracy: 0.8707 - loss: 0.36
17 - val_accuracy: 0.8518 - val_loss: 0.4339

Epoch 13/20
1688/1688  2s 1ms/step - accuracy: 0.8704 - loss: 0.36
55 - val_accuracy: 0.8590 - val_loss: 0.4431

Epoch 14/20
1688/1688  2s 1ms/step - accuracy: 0.8734 - loss: 0.35
20 - val_accuracy: 0.8598 - val_loss: 0.4485

Epoch 15/20
1688/1688  2s 1ms/step - accuracy: 0.8736 - loss: 0.35
31 - val_accuracy: 0.8607 - val_loss: 0.4659

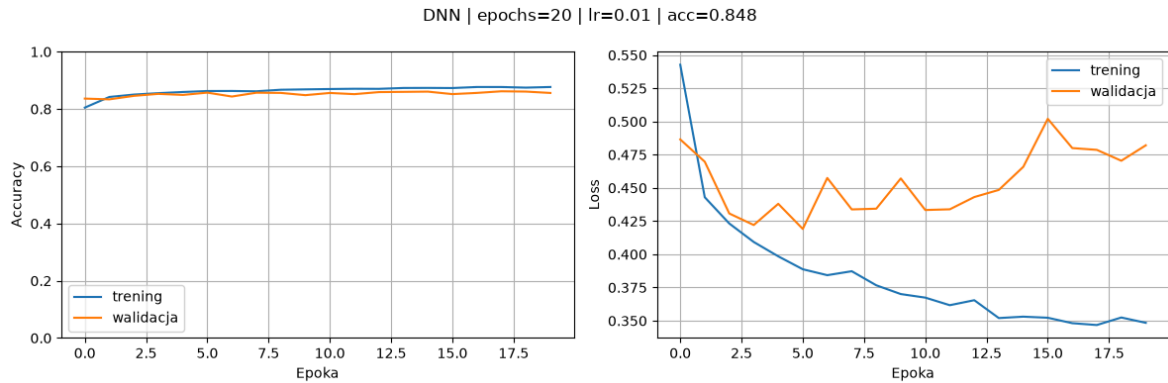
Epoch 16/20
1688/1688  2s 1ms/step - accuracy: 0.8732 - loss: 0.35
22 - val_accuracy: 0.8520 - val_loss: 0.5020

Epoch 17/20
1688/1688  2s 1ms/step - accuracy: 0.8768 - loss: 0.34
82 - val_accuracy: 0.8562 - val_loss: 0.4800

Epoch 18/20
1688/1688  2s 1ms/step - accuracy: 0.8768 - loss: 0.34
68 - val_accuracy: 0.8618 - val_loss: 0.4787

Epoch 19/20
1688/1688  2s 1ms/step - accuracy: 0.8746 - loss: 0.35
24 - val_accuracy: 0.8608 - val_loss: 0.4705

Epoch 20/20
1688/1688  2s 1ms/step - accuracy: 0.8768 - loss: 0.34
84 - val_accuracy: 0.8560 - val_loss: 0.4821



313/313 ————— 0s 503us/step
Zapisano 1525 błędów -> wrong_predictions/DNN_epochs=20_lr=0.01/
Model: "sequential_8"

Layer (type)	Output Shape	
flatten_8 (Flatten)	(None, 784)	
dense_10 (Dense)	(None, 128)	
dense_11 (Dense)	(None, 64)	
dense_12 (Dense)	(None, 10)	



Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688  3s 1ms/step - accuracy: 0.8170 - loss: 0.51
11 - val_accuracy: 0.8525 - val_loss: 0.4073

Epoch 2/20
1688/1688  2s 1ms/step - accuracy: 0.8643 - loss: 0.37
49 - val_accuracy: 0.8567 - val_loss: 0.3837

Epoch 3/20
1688/1688  2s 1ms/step - accuracy: 0.8752 - loss: 0.33
77 - val_accuracy: 0.8612 - val_loss: 0.3780

Epoch 4/20
1688/1688  2s 1ms/step - accuracy: 0.8842 - loss: 0.31
19 - val_accuracy: 0.8672 - val_loss: 0.3648

Epoch 5/20
1688/1688  2s 1ms/step - accuracy: 0.8912 - loss: 0.29
28 - val_accuracy: 0.8690 - val_loss: 0.3583

Epoch 6/20
1688/1688  2s 1ms/step - accuracy: 0.8960 - loss: 0.27
79 - val_accuracy: 0.8732 - val_loss: 0.3537

Epoch 7/20
1688/1688  2s 1ms/step - accuracy: 0.9008 - loss: 0.26
40 - val_accuracy: 0.8762 - val_loss: 0.3497

Epoch 8/20
1688/1688  2s 1ms/step - accuracy: 0.9065 - loss: 0.25
01 - val_accuracy: 0.8782 - val_loss: 0.3518

Epoch 9/20
1688/1688  2s 1ms/step - accuracy: 0.9088 - loss: 0.23
95 - val_accuracy: 0.8770 - val_loss: 0.3646

Epoch 10/20
1688/1688  2s 1ms/step - accuracy: 0.9124 - loss: 0.23
05 - val_accuracy: 0.8758 - val_loss: 0.3739

Epoch 11/20
1688/1688  2s 1ms/step - accuracy: 0.9175 - loss: 0.22
06 - val_accuracy: 0.8835 - val_loss: 0.3710

Epoch 12/20
1688/1688  2s 1ms/step - accuracy: 0.9181 - loss: 0.21
56 - val_accuracy: 0.8790 - val_loss: 0.3693

Epoch 13/20
1688/1688  2s 1ms/step - accuracy: 0.9232 - loss: 0.20
44 - val_accuracy: 0.8782 - val_loss: 0.3830

Epoch 14/20
1688/1688  2s 1ms/step - accuracy: 0.9250 - loss: 0.19
72 - val_accuracy: 0.8765 - val_loss: 0.4107

Epoch 15/20
1688/1688  2s 1ms/step - accuracy: 0.9280 - loss: 0.19
07 - val_accuracy: 0.8802 - val_loss: 0.4062

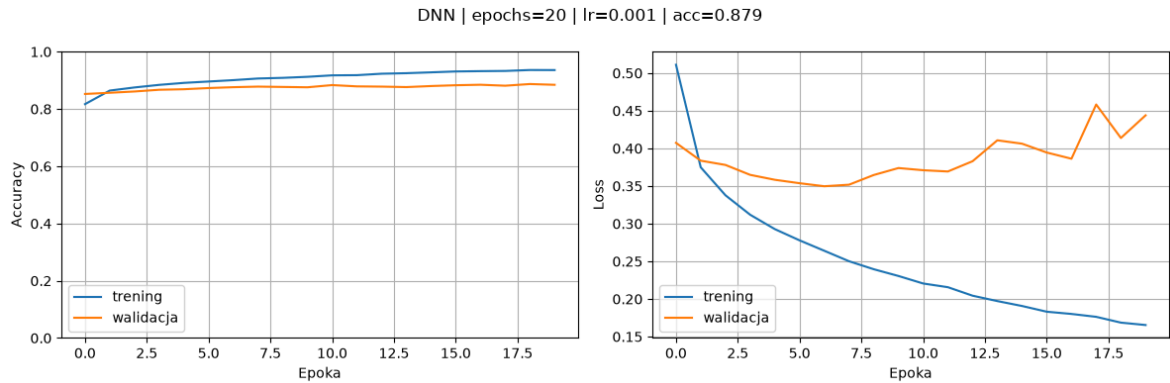
Epoch 16/20
1688/1688  2s 1ms/step - accuracy: 0.9311 - loss: 0.18
31 - val_accuracy: 0.8830 - val_loss: 0.3945

Epoch 17/20
1688/1688  2s 1ms/step - accuracy: 0.9322 - loss: 0.18
01 - val_accuracy: 0.8847 - val_loss: 0.3863

Epoch 18/20
1688/1688  3s 1ms/step - accuracy: 0.9329 - loss: 0.17
63 - val_accuracy: 0.8813 - val_loss: 0.4583

Epoch 19/20
1688/1688  2s 1ms/step - accuracy: 0.9361 - loss: 0.16
86 - val_accuracy: 0.8872 - val_loss: 0.4138

Epoch 20/20
1688/1688  2s 1ms/step - accuracy: 0.9359 - loss: 0.16
54 - val_accuracy: 0.8845 - val_loss: 0.4439



313/313 ————— 0s 498us/step
Zapisano 1206 błędów -> wrong_predictions/DNN_epochs=20_lr=0.001/
Model: "sequential_9"

Layer (type)	Output Shape	
flatten_9 (Flatten)	(None, 784)	
dense_13 (Dense)	(None, 128)	
dense_14 (Dense)	(None, 64)	
dense_15 (Dense)	(None, 10)	



Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1688/1688  3s 1ms/step - accuracy: 0.7763 - loss: 0.69
50 - val_accuracy: 0.8282 - val_loss: 0.4881

Epoch 2/20
1688/1688  2s 1ms/step - accuracy: 0.8456 - loss: 0.44
74 - val_accuracy: 0.8437 - val_loss: 0.4320

Epoch 3/20
1688/1688  2s 1ms/step - accuracy: 0.8599 - loss: 0.40
43 - val_accuracy: 0.8553 - val_loss: 0.4030

Epoch 4/20
1688/1688  2s 1ms/step - accuracy: 0.8681 - loss: 0.37
87 - val_accuracy: 0.8593 - val_loss: 0.3865

Epoch 5/20
1688/1688  2s 1ms/step - accuracy: 0.8740 - loss: 0.36
03 - val_accuracy: 0.8635 - val_loss: 0.3737

Epoch 6/20
1688/1688  2s 1ms/step - accuracy: 0.8784 - loss: 0.34
58 - val_accuracy: 0.8688 - val_loss: 0.3650

Epoch 7/20
1688/1688  2s 1ms/step - accuracy: 0.8822 - loss: 0.33
36 - val_accuracy: 0.8690 - val_loss: 0.3576

Epoch 8/20
1688/1688  2s 1ms/step - accuracy: 0.8856 - loss: 0.32
31 - val_accuracy: 0.8718 - val_loss: 0.3516

Epoch 9/20
1688/1688  2s 1ms/step - accuracy: 0.8890 - loss: 0.31
38 - val_accuracy: 0.8740 - val_loss: 0.3462

Epoch 10/20
1688/1688  2s 1ms/step - accuracy: 0.8911 - loss: 0.30
53 - val_accuracy: 0.8737 - val_loss: 0.3417

Epoch 11/20
1688/1688  2s 1ms/step - accuracy: 0.8938 - loss: 0.29
77 - val_accuracy: 0.8740 - val_loss: 0.3386

Epoch 12/20
1688/1688  2s 1ms/step - accuracy: 0.8959 - loss: 0.29
06 - val_accuracy: 0.8758 - val_loss: 0.3358

Epoch 13/20
1688/1688  2s 1ms/step - accuracy: 0.8981 - loss: 0.28
40 - val_accuracy: 0.8773 - val_loss: 0.3331

Epoch 14/20
1688/1688  2s 1ms/step - accuracy: 0.9005 - loss: 0.27
80 - val_accuracy: 0.8783 - val_loss: 0.3307

Epoch 15/20
1688/1688  2s 1ms/step - accuracy: 0.9023 - loss: 0.27
23 - val_accuracy: 0.8802 - val_loss: 0.3285

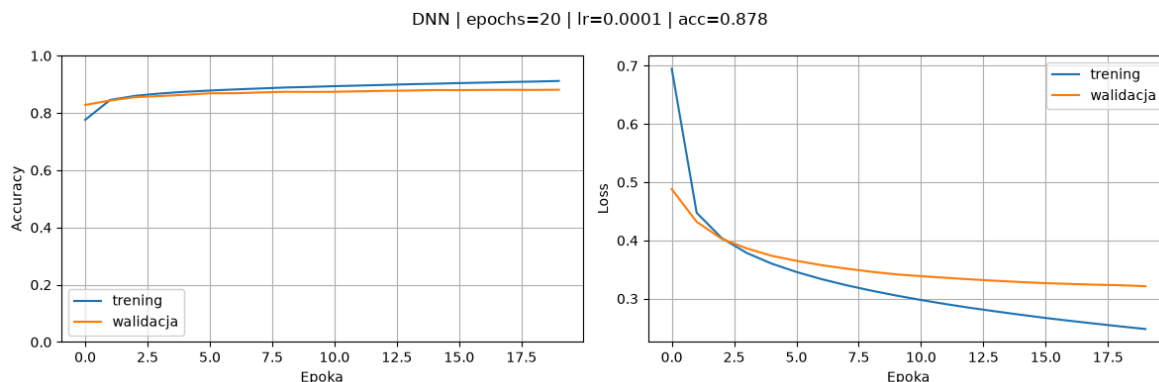
Epoch 16/20
1688/1688  2s 1ms/step - accuracy: 0.9045 - loss: 0.26
68 - val_accuracy: 0.8798 - val_loss: 0.3265

Epoch 17/20
1688/1688  2s 1ms/step - accuracy: 0.9062 - loss: 0.26
17 - val_accuracy: 0.8805 - val_loss: 0.3252

Epoch 18/20
1688/1688  2s 1ms/step - accuracy: 0.9082 - loss: 0.25
69 - val_accuracy: 0.8808 - val_loss: 0.3238

Epoch 19/20
1688/1688  2s 1ms/step - accuracy: 0.9098 - loss: 0.25
23 - val_accuracy: 0.8805 - val_loss: 0.3230

Epoch 20/20
1688/1688  2s 1ms/step - accuracy: 0.9119 - loss: 0.24
78 - val_accuracy: 0.8813 - val_loss: 0.3214



313/313 ————— 0s 515us/step

Zapisano 1217 błędów -> wrong_predictions/DNN_epochs=20_lr=0.0001/

Analiza wyników klasyfikacji

Rezultaty zebrane z przeprowadzonych eksperymentów zostały skondensowane do jednej tabeli, gdzie modele posortowano ze względu na wartość wskaźnika accuracy (precyzja). Ponadto, duplikaty zostały usunięte.

```
In [40]: summary = pd.DataFrame(all_results)
summary = summary.drop_duplicates(subset=["Model", "Epochs", "Learning rate"])
summary = summary.sort_values("Accuracy", ascending=False).reset_index(drop=True)
summary
```

Out[40]:

	Model	Epochs	Learning rate	Accuracy
0	DNN	20	0.0010	0.8794
1	DNN	20	0.0001	0.8783
2	DNN	20	0.0100	0.8475
3	Liniowy	20	0.0010	0.8442
4	Liniowy	10	0.0010	0.8435
5	Liniowy	50	0.0010	0.8429
6	Liniowy	20	0.0001	0.8394
7	Liniowy	20	0.0100	0.8201

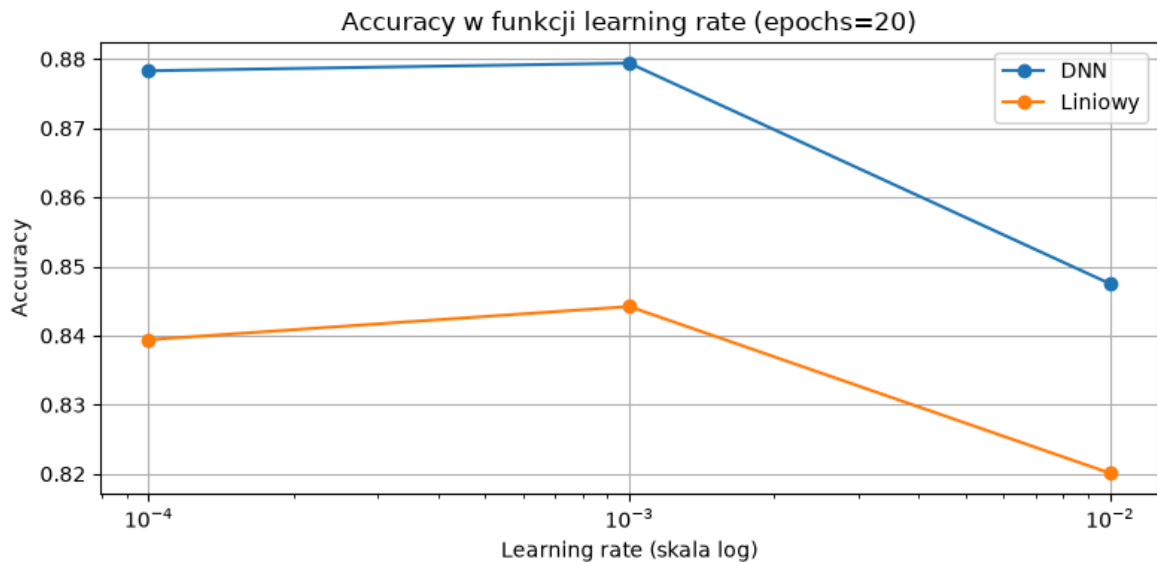
Pierwszy wykres zbiorczy (pokazany poniżej), przedstawia wykres wartości **accuracy** w funkcji **learning rate** dla obu modeli - liniowego oraz nieliniowego. Jak można zauważyć zarówno dla modelu liniowego jak i nieliniowego ukazuje się wyraźna górką dla wartości **learning rate** równej **0.001**, co oznacza że wartość wyższa niż ta wskazuje problem z tzw. overfittingiem, czyli przetrenowaniem sieci, natomiast wartość mniejsza jest powoduje mniej efektywne uczenie.

```
In [41]: # Zbiorczy wykres accuracy w funkcji learning rate dla obu modeli
# Dane: eksperymenty z epochs=20, zmienny lr
lr_results = summary[summary["Epochs"] == 20].copy()

plt.figure(figsize=(8, 4))
for model_name, group in lr_results.groupby("Model"):
    group_sorted = group.sort_values("Learning rate")
```

```
plt.plot(group_sorted["Learning rate"], group_sorted["Accuracy"],
         marker="o", label=model_name)

plt.xscale("log")
plt.xlabel("Learning rate (skala log)")
plt.ylabel("Accuracy")
plt.title("Accuracy w funkcji learning rate (epochs=20)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

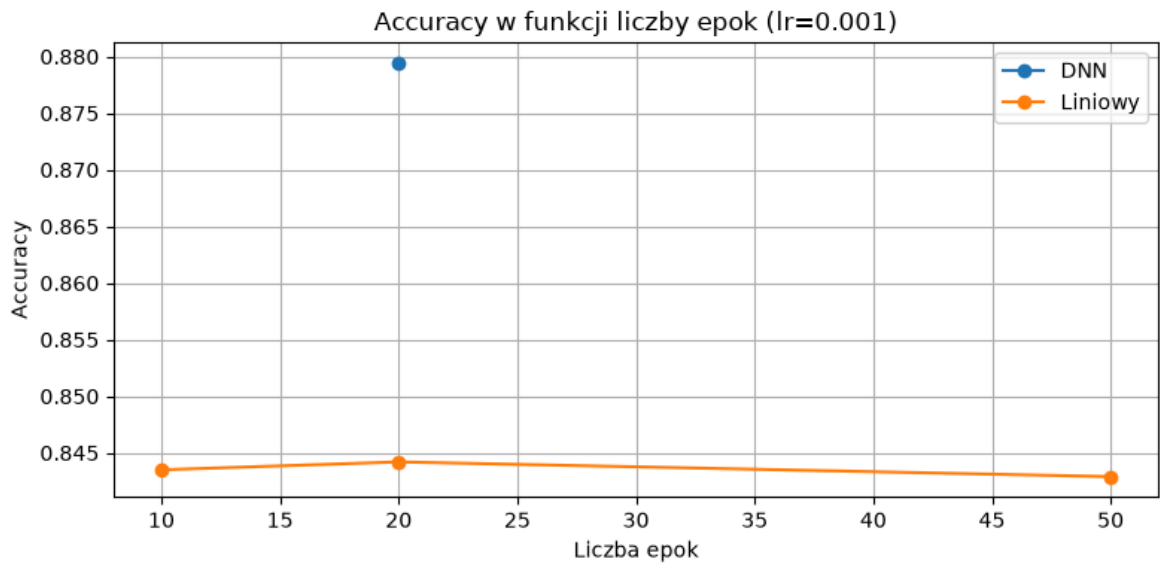


Drugi wykres zbiorczy przedstawia charakterystykę accuracy w funkcji epok, czyli jak ilość epok wpływa na **accuracy** przy stałym **learning rate**. Tutaj dla modelu nieliniowego i liniowego wnioski są inne - dla modelu nieliniowego im większa ilość epok tym wskaźnik accuracy jest większy, natomiast dla sieci liniowej różnice są ledwo zauważalne.

```
In [42]: # Zbiorczy wykres accuracy w funkcji liczby epok dla obu modeli
# Dane: eksperymenty z lr=0.001, zmienna liczba epok
epoch_results = summary[summary["Learning rate"] == 0.001].copy()

plt.figure(figsize=(8, 4))
for model_name, group in epoch_results.groupby("Model"):
    group_sorted = group.sort_values("Epochs")
    plt.plot(group_sorted["Epochs"], group_sorted["Accuracy"],
             marker="o", label=model_name)

plt.xlabel("Liczba epok")
plt.ylabel("Accuracy")
plt.title("Accuracy w funkcji liczby epok (lr=0.001)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [43]: # Trening finalnego modelu na podstawie najlepszych parametrów z eksperymentu
best_model = build_dnn_model(lr=0.001)
best_model.fit(
    train_images_normalized, train_labels,
    epochs=50,
    validation_split=0.1,
    verbose=1
)

# Predykcje
y_pred_proba = best_model.predict(test_images_normalized)
y_pred = np.argmax(y_pred_proba, axis=1)
```

Epoch 1/50
1688/1688 ————— 3s 1ms/step - accuracy: 0.8203 - loss: 0.50
52 - val_accuracy: 0.8533 - val_loss: 0.4016
Epoch 2/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.8651 - loss: 0.37
21 - val_accuracy: 0.8607 - val_loss: 0.3678
Epoch 3/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.8773 - loss: 0.33
43 - val_accuracy: 0.8690 - val_loss: 0.3536
Epoch 4/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.8854 - loss: 0.30
93 - val_accuracy: 0.8658 - val_loss: 0.3592
Epoch 5/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.8916 - loss: 0.29
17 - val_accuracy: 0.8677 - val_loss: 0.3680
Epoch 6/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.8964 - loss: 0.27
58 - val_accuracy: 0.8715 - val_loss: 0.3569
Epoch 7/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9015 - loss: 0.26
24 - val_accuracy: 0.8748 - val_loss: 0.3611
Epoch 8/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9056 - loss: 0.25
11 - val_accuracy: 0.8682 - val_loss: 0.3874
Epoch 9/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9099 - loss: 0.24
12 - val_accuracy: 0.8773 - val_loss: 0.3657
Epoch 10/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9139 - loss: 0.23
15 - val_accuracy: 0.8728 - val_loss: 0.3789
Epoch 11/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9170 - loss: 0.22
26 - val_accuracy: 0.8758 - val_loss: 0.3840
Epoch 12/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9181 - loss: 0.21
50 - val_accuracy: 0.8732 - val_loss: 0.3907
Epoch 13/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9224 - loss: 0.20
62 - val_accuracy: 0.8763 - val_loss: 0.4005
Epoch 14/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9254 - loss: 0.19
81 - val_accuracy: 0.8820 - val_loss: 0.3916
Epoch 15/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9279 - loss: 0.19
21 - val_accuracy: 0.8817 - val_loss: 0.3949
Epoch 16/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9296 - loss: 0.18
63 - val_accuracy: 0.8795 - val_loss: 0.4067
Epoch 17/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9322 - loss: 0.17
95 - val_accuracy: 0.8737 - val_loss: 0.4462
Epoch 18/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9344 - loss: 0.17
54 - val_accuracy: 0.8763 - val_loss: 0.4173
Epoch 19/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9363 - loss: 0.17
17 - val_accuracy: 0.8722 - val_loss: 0.4307
Epoch 20/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9380 - loss: 0.16
52 - val_accuracy: 0.8782 - val_loss: 0.4425

Epoch 21/50
1688/1688  2s 1ms/step - accuracy: 0.9393 - loss: 0.15
99 - val_accuracy: 0.8825 - val_loss: 0.4370
Epoch 22/50
1688/1688  2s 1ms/step - accuracy: 0.9404 - loss: 0.15
52 - val_accuracy: 0.8803 - val_loss: 0.4435
Epoch 23/50
1688/1688  2s 1ms/step - accuracy: 0.9428 - loss: 0.15
10 - val_accuracy: 0.8750 - val_loss: 0.4855
Epoch 24/50
1688/1688  2s 1ms/step - accuracy: 0.9440 - loss: 0.14
91 - val_accuracy: 0.8752 - val_loss: 0.4948
Epoch 25/50
1688/1688  2s 1ms/step - accuracy: 0.9457 - loss: 0.14
54 - val_accuracy: 0.8673 - val_loss: 0.5286
Epoch 26/50
1688/1688  2s 1ms/step - accuracy: 0.9456 - loss: 0.14
44 - val_accuracy: 0.8842 - val_loss: 0.4538
Epoch 27/50
1688/1688  2s 1ms/step - accuracy: 0.9465 - loss: 0.14
13 - val_accuracy: 0.8770 - val_loss: 0.4990
Epoch 28/50
1688/1688  2s 1ms/step - accuracy: 0.9482 - loss: 0.13
76 - val_accuracy: 0.8793 - val_loss: 0.4883
Epoch 29/50
1688/1688  2s 1ms/step - accuracy: 0.9508 - loss: 0.13
47 - val_accuracy: 0.8812 - val_loss: 0.4926
Epoch 30/50
1688/1688  2s 1ms/step - accuracy: 0.9507 - loss: 0.13
01 - val_accuracy: 0.8810 - val_loss: 0.5093
Epoch 31/50
1688/1688  2s 1ms/step - accuracy: 0.9519 - loss: 0.12
83 - val_accuracy: 0.8817 - val_loss: 0.5682
Epoch 32/50
1688/1688  2s 1ms/step - accuracy: 0.9540 - loss: 0.12
26 - val_accuracy: 0.8828 - val_loss: 0.5249
Epoch 33/50
1688/1688  2s 1ms/step - accuracy: 0.9536 - loss: 0.12
34 - val_accuracy: 0.8840 - val_loss: 0.5604
Epoch 34/50
1688/1688  2s 1ms/step - accuracy: 0.9538 - loss: 0.12
21 - val_accuracy: 0.8822 - val_loss: 0.5181
Epoch 35/50
1688/1688  2s 1ms/step - accuracy: 0.9554 - loss: 0.12
03 - val_accuracy: 0.8820 - val_loss: 0.5569
Epoch 36/50
1688/1688  2s 1ms/step - accuracy: 0.9576 - loss: 0.11
43 - val_accuracy: 0.8848 - val_loss: 0.5526
Epoch 37/50
1688/1688  2s 1ms/step - accuracy: 0.9573 - loss: 0.11
33 - val_accuracy: 0.8780 - val_loss: 0.6053
Epoch 38/50
1688/1688  2s 1ms/step - accuracy: 0.9572 - loss: 0.11
41 - val_accuracy: 0.8793 - val_loss: 0.6140
Epoch 39/50
1688/1688  2s 1ms/step - accuracy: 0.9579 - loss: 0.11
04 - val_accuracy: 0.8800 - val_loss: 0.5992
Epoch 40/50
1688/1688  2s 1ms/step - accuracy: 0.9591 - loss: 0.10
89 - val_accuracy: 0.8793 - val_loss: 0.5920

```

Epoch 41/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9605 - loss: 0.10
42 - val_accuracy: 0.8803 - val_loss: 0.6291
Epoch 42/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9600 - loss: 0.10
84 - val_accuracy: 0.8773 - val_loss: 0.6615
Epoch 43/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9626 - loss: 0.09
98 - val_accuracy: 0.8765 - val_loss: 0.6362
Epoch 44/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9619 - loss: 0.10
10 - val_accuracy: 0.8792 - val_loss: 0.6786
Epoch 45/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9616 - loss: 0.10
20 - val_accuracy: 0.8768 - val_loss: 0.6598
Epoch 46/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9630 - loss: 0.09
70 - val_accuracy: 0.8810 - val_loss: 0.6284
Epoch 47/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9629 - loss: 0.09
67 - val_accuracy: 0.8772 - val_loss: 0.6623
Epoch 48/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9641 - loss: 0.09
42 - val_accuracy: 0.8808 - val_loss: 0.6572
Epoch 49/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9652 - loss: 0.09
12 - val_accuracy: 0.8835 - val_loss: 0.6745
Epoch 50/50
1688/1688 ————— 2s 1ms/step - accuracy: 0.9655 - loss: 0.09
22 - val_accuracy: 0.8830 - val_loss: 0.6526
313/313 ————— 0s 585us/step

```

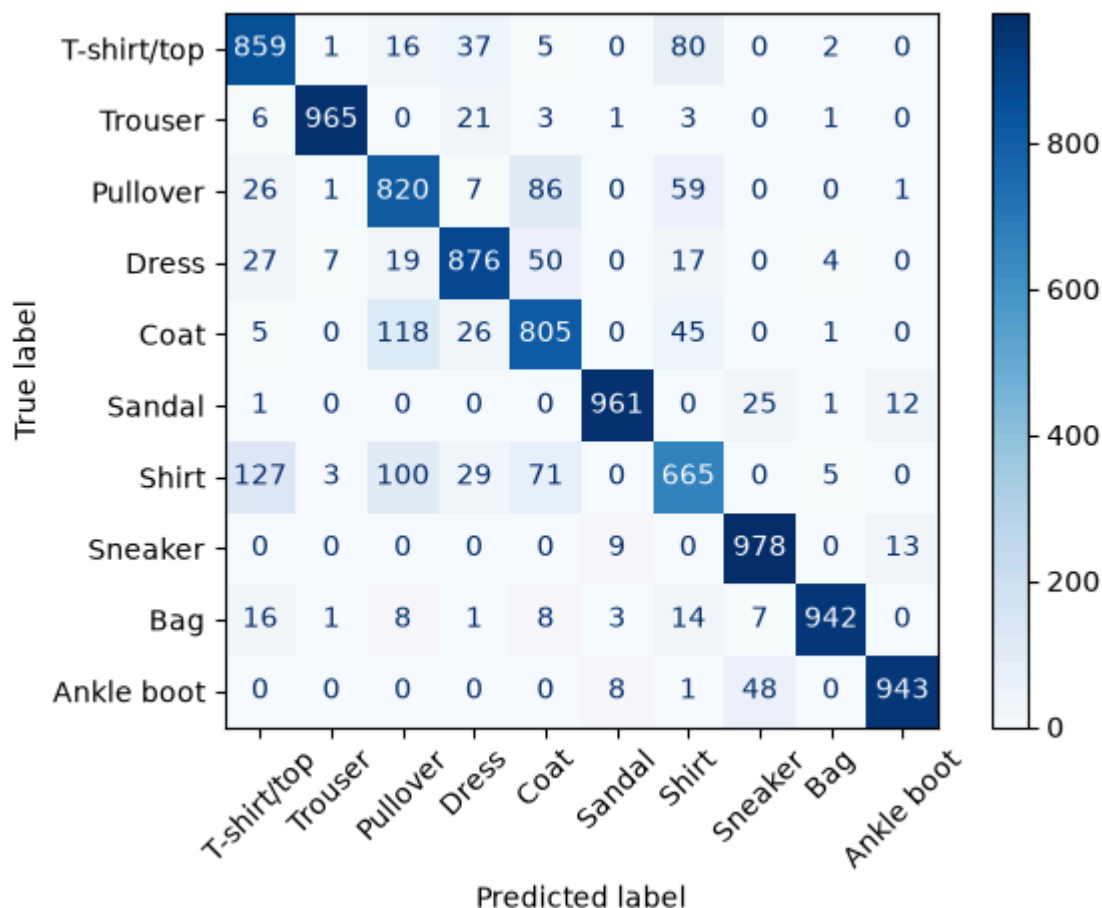
W tym miejscu w ramach dokładniejszej analizy otrzymanych wyników został utworzony tzw. confusion matrix, czyli macierz pomyłek. Po przekątnej widać ile z próbek zostało poprawnie zaklasyfikowanych a na pozostałych polach są widoczne błędne klasyfikacje. Na podstawie tejże macierzy pomyłek widać, że największe problemy w klasyfikacji sprawiły głównie trzy klasy: **Shirt**, **Pullover** oraz **Coat**.

```

In [45]: from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    test_labels, y_pred,
    display_labels=class_names,
    xticks_rotation=45,
    cmap="Blues"
)
plt.tight_layout()
plt.show()

```

Poniżej w ramach daleszej analizy zostały pokazane wartości parametrów precision, recall i F1 dla każdej z klas. Jak widać, wartości wszystkich klas oprócz **Shirt** są na ogół dosyć wysokie.

Poszczególne wartości w praktyce oznaczają:

- **Accuracy (Dokładność ogólna):** określa stosunek poprawnie sklasyfikowanych obrazów do wszystkich próbek w zbiorze testowym. Była to główna metryka monitorowana podczas treningu (funkcja `fit`), pozwalająca ocenić ogólny postęp nauki modeli na przestrzeni epok.
- **Precision (Precyzja):** określa zdolność modelu do nieoznaczania klasy negatywnej jako pozytywnej (tj. jak duży procent ubrań zaklasyfikowanych do danej kategorii faktycznie do niej należał).
- **Recall (Czułość / Pełność):** mierzy zdolność modelu do wykrywania wszystkich próbek z danej klasy (tj. jaki procent ubrań z danej kategorii został prawidłowo wykryty).
- **F1-score**- średnia harmoniczna precyzji i czułości, stanowiąca zbalansowany wskaźnik efektywności modelu dla każdej klasy osobno.
- **Confusion Matrix (Macierz pomyłek):** kluczowe narzędzie analityczne, które pozwoliło na szczegółową identyfikację, z którymi klasami (np. **Shirt**, **Pullover**, **Coat**) modele miały największy problem z powodu ich wizualnego podobieństwa w niskiej rozdzielczości 28×28 pikseli.

```
In [46]: from sklearn.metrics import classification_report
```

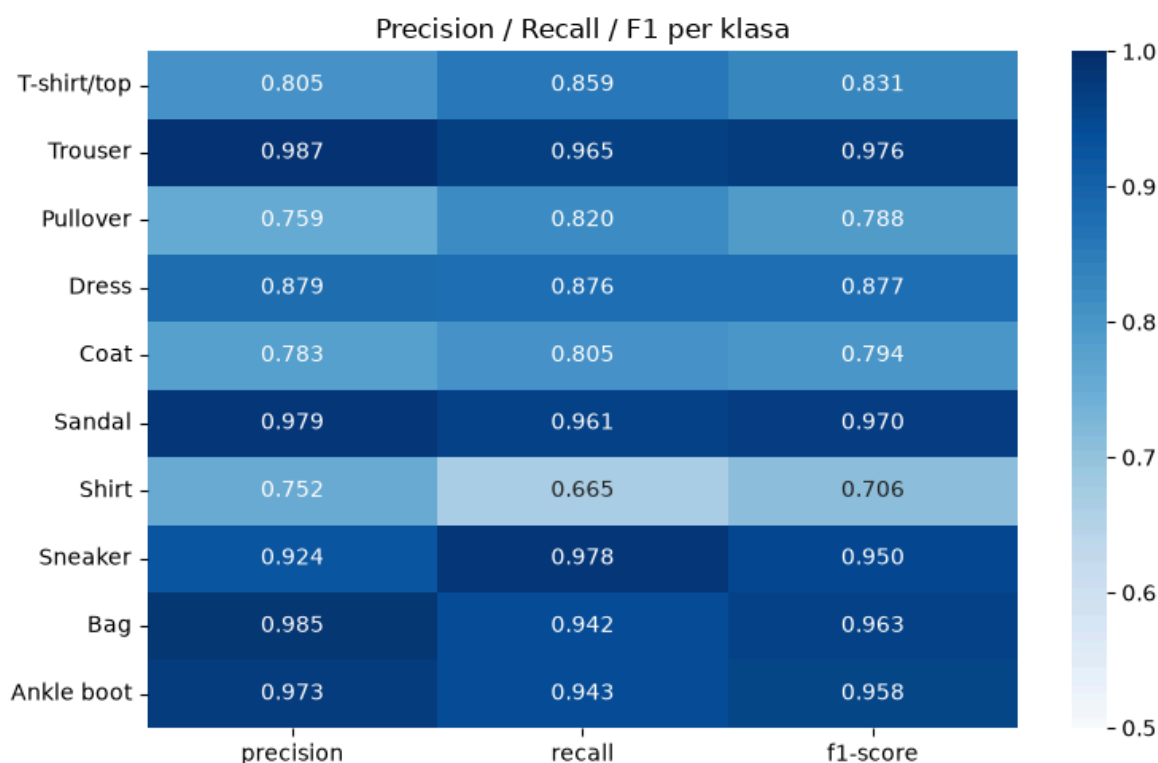
```

report = classification_report(
    test_labels, y_pred,
    target_names=class_names,
    output_dict=True
)

# Wizualizacja jako heatmapa – czytelniejsza niż surowy tekst
report_df = pd.DataFrame(report).T.iloc[:-3, :3] # tylko klasy, bez avg
report_df = report_df.astype(float).round(3)

plt.figure(figsize=(8, 5))
sns.heatmap(report_df, annot=True, fmt=".3f", cmap="Blues", vmin=0.5, vma
plt.title("Precision / Recall / F1 per klasa")
plt.tight_layout()
plt.show()

```



Dla dodatkowej wizualizacji w tym miejscu zostają przedstawione przykładowe obrazy, które zostały poddane błędnej klasyfikacji.

```

In [47]: wrong_idx = np.where(y_pred != test_labels)[0]

fig, axes = plt.subplots(3, 5, figsize=(14, 9))
fig.suptitle("Przykłady błędnych klasyfikacji")

for ax, idx in zip(axes.flat, wrong_idx[:15]):
    ax.imshow(test_images_normalized[idx], cmap="binary")
    ax.set_title(
        f"{class_names[test_labels[idx]]} → {class_names[y_pred[idx]]}\n"
        f"pewność: {y_pred_proba[idx, y_pred[idx]]:.2f}",
        fontsize=8
    )
    ax.axis("off")

plt.tight_layout()
plt.show()

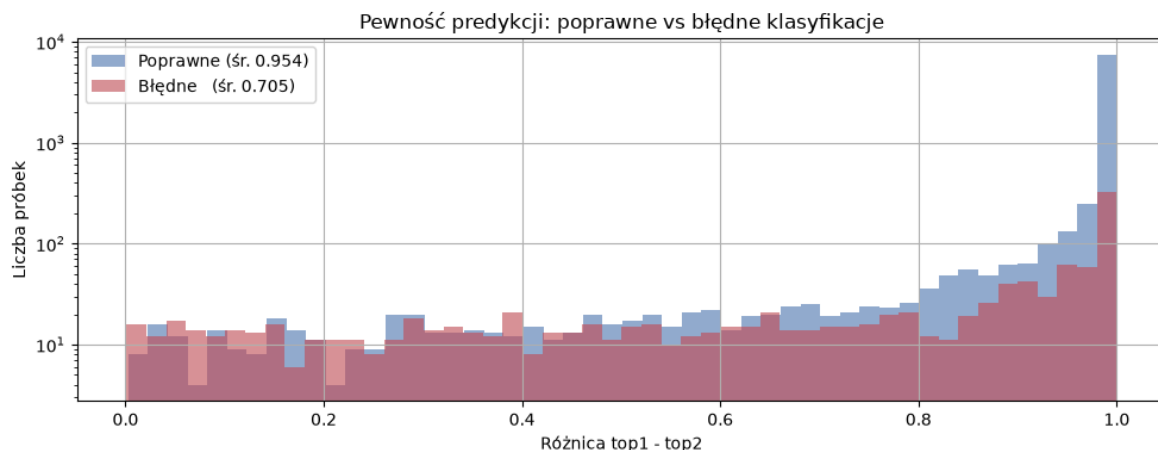
```



Ostatni wykres przedstawia pewność predykcji w formie różnicy wartości `top1` i `top2`. W praktyce oznacza to z jaką "pewnością" model zaklasyfikował daną próbkę do danej klasy. Na wykresie widać, że obie wartości są skośne lewostronnie, co oznacza że model był zarówno pewny poprawnych klasyfikacji jak i tych niepoprawnych.

```
In [48]: # Różnica między najwyższym a drugim prawdopodobieństwem – miara pewności
sorted_proba = np.sort(y_pred_proba, axis=1)[::-1]
margin = sorted_proba[:, 0] - sorted_proba[:, 1]
correct_mask = (y_pred == test_labels)

plt.figure(figsize=(10, 4))
plt.hist(margin[correct_mask], bins=50, alpha=0.6, label=f"Poprawne (śr.
plt.hist(margin[~correct_mask], bins=50, alpha=0.6, label=f"Błędne (śr.
plt.yscale("log")
plt.xlabel("Różnica top1 - top2")
plt.ylabel("Liczba próbek")
plt.title("Pewność predykcji: poprawne vs błędne klasyfikacje")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Podsumowanie i wnioski

Celem projektu była klasyfikacja obrazów z datasetu Fashion MNIST z wykorzystaniem biblioteki Keras, ze szczególnym naciskiem na analizę liniowej sieci neuronowej oraz jej porównanie z głęboką siecią nieliniową (DNN).

Przeprowadzone eksperymenty wykazały, że dla obu architektur optymalną wartością współczynnika uczenia okazało się `lr = 0.001`. Wartość wyższa (`lr = 0.01`) prowadziła do niestabilnego treningu i objawów overfittingu widocznych na krzywych funkcji straty - `val_loss` przestawał maleć lub zaczynał rosnąć podczas gdy `loss` treningowy nadal się obniżał. Wartość niższa (`lr = 0.0001`) skutkowała wolniejszą i mniej efektywną konwergencją, co przekładało się na niższe końcowe `accuracy` przy tej samej liczbie epok.

Wpływ liczby epok. Dla modelu liniowego zwiększanie liczby epok powyżej 20 przynosiło marginalne korzyści - model osiągał swój sufit możliwości stosunkowo szybko, co jest bezpośrednią konsekwencją jego architektury. Brak warstw ukrytych z nieliniową aktywacją ogranicza zdolność modelu do wychwytywania złożonych wzorców w danych, przez co dodatkowe epoki nie wnoszą istotnej poprawy. Model DNN natomiast konsekwentnie poprawiał `accuracy` wraz ze wzrostem liczby epok, osiągając najlepsze wyniki przy `epochs = 50`.

Analiza błędów klasyfikacji. Macierz pomyłek oraz raport `precision/recall/F1` ujawniły że najtrudniejszymi klasami do poprawnej klasyfikacji były `Shirt`, `Pullover` oraz `Coat`. Przyczyną jest ich wizualne podobieństwo - wszystkie trzy klasy mają zbliżony obrys sylwetki, podobną strefę kołnierza i długość rękawów, a rozdzielczość `28x28` pikseli nie dostarcza wystarczającej ilości szczegółów do pewnego rozróżnienia. Klasy takie jak `Trouser` i `Bag` były natomiast klasyfikowane niemal bezbłędnie ze względu na unikalny i charakterystyczny kształt.